

Teil4: Codeerzeugung RISC-V

Erzeugen Sie aus der Zwischendarstellung Maschinencode für den RISC-V. Der Maschinencode wird ohne zusätzliche Daten in eine Datei (*.iex) gespeichert, da in MIEC nur temporäre Daten möglich sind, die nicht initialisierbar sind.

Der Generator für den Maschinencode RISC-V (`CodeGenRISCV.h` und `CodeGenRISCV.cpp`) wird zur Verfügung gestellt. Das Interface in `CodeGen.h` liefert eine abstrakte Schnittstelle und wird vom `CodeGenRISCV` implementiert. Diese Schnittstelle beinhaltet die wichtigsten Maschinencode-Befehle und ist gleichzeitig die Basis für weitere Codegeneratoren, die einen anderen Maschinencode als RISC-V erzeugen.

Jede implementierte Methode der Schnittstelle erzeugt den zugehörigen Operationscode (RISC-V Code) und schreibt diesen in ein Byte-Feld. Am Ende wird der gesamte Maschinencode mit Hilfe einer Methode `WriteExecutable(std::ostream& iexFile)` in eine ausführbare Datei geschrieben.

Um aus der Zwischendarstellung entsprechenden Maschinencode erzeugen zu können, sind folgende Implementierungsschritte durchzuführen:

1. Implementierung einer Klasse `CodeGenerator`, die eine Liste der einzelnen DAC-Konstrukte (Zwischendarstellung) speichert und den `CodeGenRISCV` verwaltet und verwendet.
2. Eine Methode `CodeGenerator::GenerateCode()` durchläuft die DAC-Konstrukte und erzeugt für jedes Konstrukt mit Hilfe des `CodeGenRISCV` den entsprechenden RISC-V Code. Die Variablen am Datenspeicher werden nicht automatisch vorinitialisiert, dafür ist entsprechender Maschinencode zu erzeugen.
3. Implementierung eines Register-Administrators der die Register der virtuellen Maschine verwaltet. Die virtuelle Maschine stellt 32 Register zur Verfügung.

Virtuelle Maschine für den RISC-V

Benutzen Sie die virtuelle Maschine `VMRiscV`, die auf der Kommunikationsplattform zur Verfügung gestellt wird, um den Maschinencode auszuführen. Die `VMRiscV` legt Speicherbereiche für Code und Daten fest, lädt den Programmcode (iex-Datei) und führt ihn entsprechend aus.

Aufruf der VM:

```
VMRiscV ExecutableFile.iex [-v]
```

Mit der Option -v gibt die RISC-V Maschine den aktuell ausgeführten Maschinencode auf stdout aus.

Implementierung des Code-Generators

Aus zeitlichen Gründen wird ein Vorschlag für die Schnittstelle des Code-Generators zur Verfügung gestellt.

```
1 class CodeGenerator : public Object {
2 public:
3     ...
4
5     // This method generates machine code and writes it to a file
6     void GenerateCode(std::ostream& ost);
7
8 private:
9
10    // contains all unresolved jumps that occurred during code generation
11    using TUnresolvedJumps = std::list<std::pair<WORD, DACEntry::SPtr>>;
12
13    // this method resolves all jumps saved in the UnresolvedJumps list
14    void ResolveUnresolvedJumps();
15
16    // generates code with the general interface of CodeGen
17    void OperationAdd(DACEntry::SPtr pDacEntry);
18    void OperationSubtract(DACEntry::SPtr pDacEntry);
19    void OperationMultiply(DACEntry::SPtr pDacEntry);
20    void OperationDivide(DACEntry::SPtr pDacEntry);
21    void OperationAssign(DACEntry::SPtr pDacEntry);
22    void OperationJump(DACEntry::SPtr pDacEntry);
23    void OperationConditionalJump(DACEntry::SPtr relopDacEntry, DACEntry::SPtr jumpDacEntry);
24    void OperationPrint(DACEntry::SPtr pDacEntry);
25
26    // member
27    CodeGen<WORD>* mCodeGen; // code generator to use
28    DACGenerator* mDacGen;   // dacGenerator used by parser
29    RegisterAdmin* mpRegAdmin; // register admin for managing registers
30    TUnresolvedJumps mUnresolvedJumps; // list containing all unresolved jumps
31    std::vector<WORD> mStartAddresses; // vector containing start addresses for jumps
32
33    ...
34 };
```

Die Methode `GenerateCode(...)` bekommt die ausführbare Datei und durchläuft den gesamten Zwischencode. Abhängig von der Zwischencode-Anweisung ist der entsprechende Maschinencode mithilfe des Maschinencode-Generators zu erzeugen. Die Codeerzeugung für die einzelnen Operationen ist in privaten Methoden gekapselt, deren erster Parameter immer der entsprechende `DACEntry` des Zwischencodes ist.

Implementierung des Register-Administrators

Ein Register-Administrator verwaltet die Register der RISC-V Maschine. Er speichert in einer Liste die Register die während der Codeerzeugung benutzt werden bzw. frei sind. Da nur eine bestimmte Anzahl an Registern zur Verfügung steht, kann nur eine bestimmte Anzahl an temporären Zwischenergebnissen in Registern gespeichert werden. Wird die Anzahl an möglichen Zwischenergebnissen

während der Codeerzeugung überschritten, so müssen die temporären Werte auf den Registerspeicher (Stack) ausgelagert werden, um Register für weitere Berechnungen frei zu bekommen. Werden die ausgelagerten Werte in Folgeanweisungen wieder benötigt, so sind sie wieder in ein entsprechendes Register zu laden.

Die Werte von Variablen werden in der ersten Ausbaustufe nicht am Registerspeicher zwischengesichert!

Die folgende Schnittstelle eines Register-Administrators soll eine Hilfestellung bei der Implementierung liefern:

```
1 class RegisterAdmin : public Object
2 {
3 public:
4
5     ...
6
7     // returns the index of the next free register
8     RegNr GetRegister();
9
10    // returns the index of the register holding the given operand
11    // @param op:   Operand of which to get the register number
12    RegNr GetRegister(Operand::SPtr op);
13
14    // frees the register with the given number from its entry
15    // @param nr:   register number to free
16    void FreeRegister(RegNr const nr);
17
18    // saves the given entry to the register with the given number
19    // @param nr:   number of the register to write to
20    // @param etnry: DACEntry that gets saved in the register
21    void AssignRegister(RegNr const nr, DACEntry::SPtr entry);
22
23    ...
24 };
```

Die Methode `GetRegister` kann für den Variablen-Operand direkt Code erzeugen, der den Wert der Variable in ein Register lädt. Für einen Konstanten-Operanden wird Code erzeugt, der den Wert der Konstanten in das Register lädt. Das Register mit dem geladenen Wert wird zurück gegeben.

Die Methode `AssignRegister` merkt sich zu einem Register einen DAC-Operanden (entspricht dem Ergebnis einer Operation). Wird in einem Folgebefehl das Ergebnis benötigt, so kann wiederum die Methode `GetRegister(Operand::SPtr)` die Nummer des bereits verwendeten Registers mit diesem Operanden liefern.

Hinweise zur Abgabe:

- Geben Sie den gesamten Quellcode als VS-Projekt ab.
- Das gesamte Projekt ist als MIECCompiler_name1_name2.zip abzugeben. Das zip muss zusätzlich die Release-Version des MIECCompilers enthalten.
- Kommentieren Sie den Quellcode entsprechend!
- Schreiben Sie entsprechende MIEC-Testdateien und geben Sie diese mit ab.
- Die virtuelle Maschine VmRiscV ist nicht mitabzugeben.
- Das vollständige Klassendiagramm (Reverse Engineering mit EA) ist als pdf mitabzugeben.
- Die Ausgabe des Compilers (std::cout) beschränkt sich auf folgendes Format:

```
..\testfiles\ok\mixed\Test2.miec: OK
```

bzw. im Fehlerfall (std::cerr)

```
-- line 14 col 7: invalid Factor
```

```
..\testfiles\failed\TestSubtract.miec: FAILED: 1 error(s) detected
```

- Prüfen Sie die Vollständigkeit der Kommandozeilenparameter (siehe Übung1).
- Prüfen Sie das richtige Format der MIECCompiler.report Datei (siehe Übung1).
- Testen Sie Ihre Abgabe(zip-Datei) mit der automatischen Testplattform ACOTEP.
- Achten Sie auf die Einhaltung folgender Namen:
 - MIECCompiler.exe
 - MIECCompiler_name1_name2.zip
 - MIECCompiler.report
 - xfail_Datei.miec